CHAPTER 5

REACT NATIVE API, NAVIGATION, HTTP AND STORAGE

Content

- React native API
  - Alert and Geolocation
- React native Http
- React native Navigation
- React native Storage

React Native HTTP

- Many mobile apps need to load resources from a remote URL.
  - You may want to make a POST request to a REST API, or you may need to fetch a chunk of static content from another server.
- **Using Fetch**
  - React Native provides the Fetch API for your networking needs. Fetch will seem familiar if you have used XMLHttpRequest or other networking APIs before.
  - **Making requests**
    - In order to fetch content from an arbitrary URL, you can pass the URL to fetch:
    - **fetch('https://mywebsite.com/mydata.json');**

- Fetch also takes an optional second argument that allows you to customize the HTTP request. You may want to specify additional headers, or make a POST request:

```
fetch('https://mywebsite.com/endpoint/', {
  method: 'POST',
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    firstParam: 'yourValue',
    secondParam: 'yourOtherValue'
  })
});
```

- **Handling the response**
  - The above examples show how you can make a request. In many cases, you will want to do something with the response.
  - Networking is an inherently asynchronous operation. Fetch method will return a Promise that makes it straightforward to write code that works in an asynchronous manner:

```
const getMoviesFromApi = () => {
  return fetch('https://reactnative.dev/movies.json')
    .then((response) => response.json())
    .then((json) => {
      return json.movies;
    })
    .catch((error) => {
      console.error(error);
    });
};
```

- You can also use the **async** / **await** syntax in a React Native app:
  - Don't forget to catch any errors that may be thrown by fetch, otherwise they will be dropped silently.

```
const getMoviesFromApiAsync = async () => {
  try {
    const response = await fetch(
      'https://reactnative.dev/movies.json'
    );
    const json = await response.json();
    return json.movies;
  } catch (error) {
    console.error(error);
  }
};
```

How to use **async / await?**

- We use **async-await** to manage **time consuming** tasks using async await we have the option to wait for the first task before executing the second task.
  - **Async** keyword use for define **function for use await task,** when you want to use to await for wait task then **first you have to define async function.**
  - **Await** keyword use for stop execution till task response, we will use **multiple await calls** in **one async function.**

- **Syntax**
  - Async syntax **for normal function** example

```
1   async function exampleOne(items) {
2
3   }
```

  - Async syntax **for arrow function** example

```
1   const exampleSecond = async () => {
2
3   }
```

Await syntax

- You can use await function native library calls or you can also use for call your async function

```
1   await AsyncStorage.setItem("@ProductTour:key", "true");
```

- Await call own function example

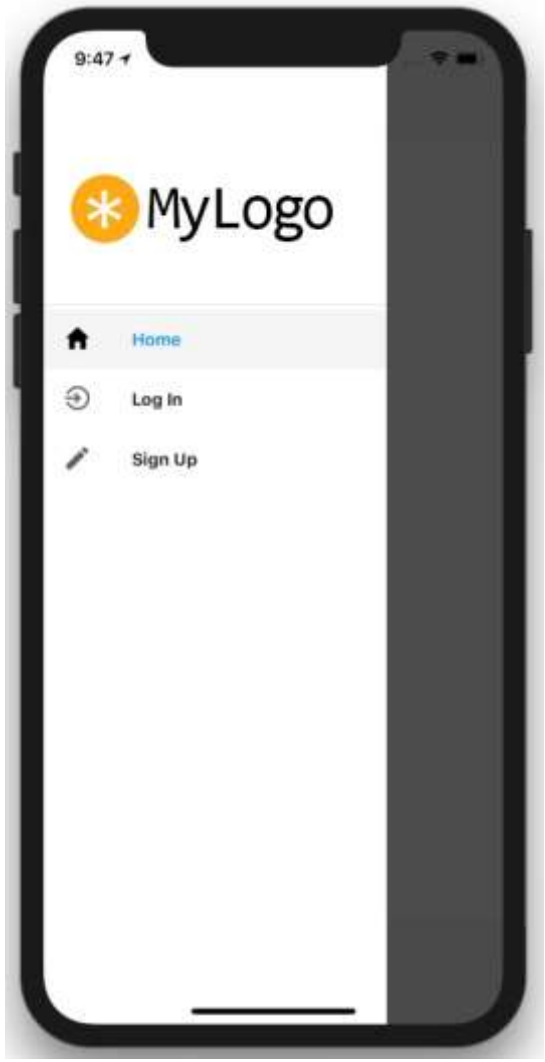
```
1   const exampleSecond = async () => {
2       await exampleOne();
3   }
```

React native Navigation

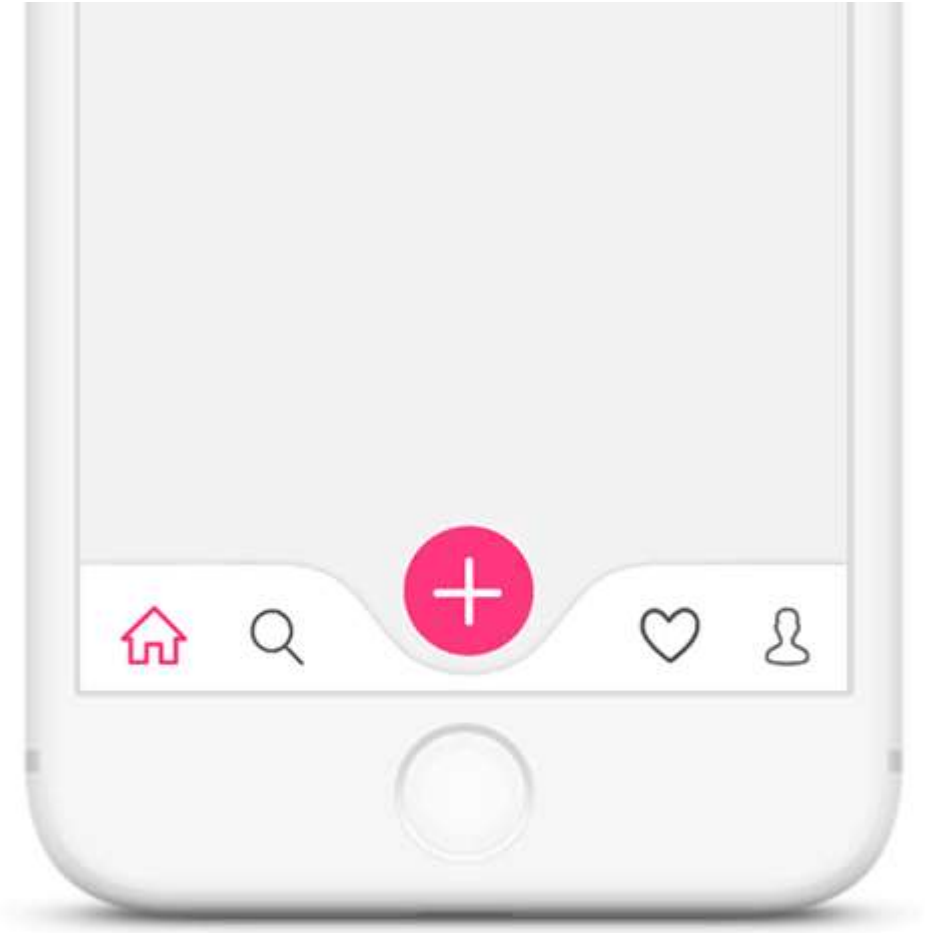
- Developing mobile applications involves many moving parts.
  - From **styling, functionality, authentication,** and **session persistence** to **navigation**
- **React Navigation** is a library consisting of components that makes building and connecting screens in React Native easy.
  - It is important to understand how **navigation in mobile applications works,** and what major differences exist between
    - **Mobile-based navigation** and
    - **Web-based navigation.**

Navigation types in React native

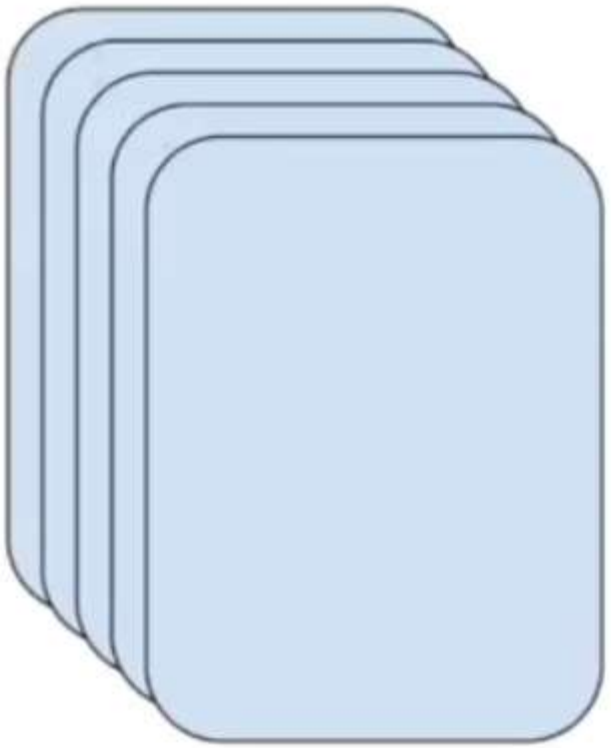
- **Drawer navigation**
  - This consists of patterns in navigation that require you to use a **drawer** from the left (sometimes right) side for navigating between and through screens.
  - Typically, it consists of links that provide a gateway to moving between screens.
    - It's similar to sidebars in web-based applications.



- **Tab navigation**
  - This is the most common form of navigation in most mobile applications.
  - Navigation is possible through **tab-based components.**
    - It consists of **bottom and top tabs.**



- **Stack navigation**
  - Transition between screens is made possible in the form of **stacks.**
  - Various screens are stacked on top of each other, and movement between screens involves replacing one screen with another.



- Most times, mobile applications make use of all types of navigators to build the various screens and functionalities of the application. This is called **nesting navigators**.

Installation and setup

- First, you need to install them in your project:
  - **npm install @react-navigation/native @react-navigation/native-stack**
- Next, install the required peer dependencies. You need to run different commands depending on whether your project is an **Expo managed project** or a **bare React Native project**.
  - If you have an Expo managed project, install the dependencies with expo:
    - ~~**expo install react-native-screens react-native-safe-area-context**~~
  - If you have a bare React Native project, install the dependencies with npm:
    - **npm install react-native-screens react-native-safe-area-context**
    - If you're on a Mac and developing for iOS, you need to install the pods (via **Cocoapods**) to complete the linking.
    - npx pod-install ios

Creating a simple stack navigation

- **Step:1- Import the following components**

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
```

- **Step:2 - Wrapping your app in NavigationContainer**
  - Now, we need to wrap the whole app in **NavigationContainer.** Usually you'd do this in your entry file, such as index.js or App.js:
    - Note: In a typical React Native app, the **NavigationContainer** should be only used once in your app at the root.

- **NavigationContainer** is a component which manages our navigation tree and contains the **navigation** state.
  - This component must wrap all navigators structure. Usually, we'd render this component at the root of our app, which is usually the component exported from App.js.

```
export default function App() {
  return (
    <NavigationContainer>

    </NavigationContainer>
  );
}
```

- **Step:3 – Creating a native stack navigation**
  - **createNativeStackNavigator** is a function that returns an object containing 2 properties:
    - **Screen** and **Navigator**.
  - Both of them are React components used for configuring the navigator.
  - The **Navigator should contain Screen elements as its children** to define the configuration for routes.

```
export default function App() {
  const Stack =createNativeStackNavigator();
  return (
```

- **Step:4 – Creating Stack Navigator**

```
<NavigationContainer>
  <Stack.Navigator>



  </Stack.Navigator>
</NavigationContainer>
```

- **Step:5 – Creating Home and Profile Screens, and import inside App**

```
export default function Home() {
  return (
    <View>
        <Text>This is Home</Text>
    </View>
  );
}
```

```
export default function Profile() {
  return (
    <View>
        <Text>This is Profile</Text>
    </View>
  );
}
```

```
import Home from './screens/Home';
import Profile from './screens/Profile';
export default function App() {
```

Step:6 – Creating Stack Screens

```
<NavigationContainer>
  <Stack.Navigator>
    <Stack.Screen name='Home' component={Home}/>
    <Stack.Screen name='Profile' component={Profile}/>
  </Stack.Navigator>
</NavigationContainer>
```

Navigating to a new screen

- We'll use the **navigation prop** that is passed down to our screen components.
  - **navigation** - the navigation prop is passed in to every **screen component** in the **native stack navigator**.
    - **navigate('Profile')** - we call the navigate function with the name of the route that we'd like to move the user to.
    - **Note**: We can only navigate to routes that have been defined on our navigator — we cannot navigate to an arbitrary component.
- So we now have a stack with two routes:
  - 1) the Home route
  - 2) the Profile route.

- **Step:7 – Update the Home Screens**

```
export default function Home({navigation}) {
  return (
    <View>
        <Text>This is Home</Text>
        <Button
            title='Go to Profile'
            onPress={()=>navigation.navigate('Profile')}/>
    </View>
  );
}
```

Navigate to a route multiple times

- If you run this code, you'll notice that when you tap "Go to Profile" that it **doesn't do anything!** This is because we are already on the Profile route.
- The navigate function roughly means "go to this screen", and if you are already on that screen then it makes sense that it would do nothing.

```
export default function Profile({navigation}) {
  return (
    <View>
        <Text>This is Profile</Text>
        <Button
            title='Go to Profile'
            onPress={()=>navigation.navigate('Profile')}
        />
    </View>
  );
}
```

- Let's suppose that we actually *want* to add another profile screen.
  - This is pretty common in cases where you pass in some unique data to each route (more on that later when we talk about params!).
  - To do this, we can change **navigate** to **push**. This allows us to express the intent to add another route regardless of the existing navigation history.

```
<Button
    title='Go to Profile'
    onPress={()=>navigation.push('Profile')}
/>
```

Going back and Go to top

- **Going back**
  - The header bar will automatically show a back button, but you can programmatically go back by calling **navigation.goBack()**.
    - On Android, the hardware back button just works as expected.
- **Go to top**
  - You can go back to an existing screen in the stack with navigation.navigate('RouteName'), and you can go back to the first screen in the stack with **navigation.popToTop()**.

Configuring the navigator

- All of the route configuration is specified as props to our **navigator**. We haven't passed any props to our navigator, so it just uses the default configuration.
- The initial route for the previous stack is the **Home** route. Try changing it to Profile by adding **initialRouteName='Profile'** props inside **Stack.Navigator** and reload the app, notice that you will now see the Profile screen.

```
<Stack.Navigator initialRouteName='Profile'>
  <Stack.Screen name="Home" component={Home}/>
  <Stack.Screen name="Profile" component={Profile}/>
</Stack.Navigator>
```

Specifying options

- Each screen in the navigator can specify some options for the navigator, such as the title to render in the header. These options can be passed in the options prop for each screen component:

```
<Stack.Screen
  name="Home"
  component={Home}
  options={{ title: 'My Home Page' }}
/>
```

- For common options, we can pass **screenOptions** to **Stack.Navigator**

Passing parameters to routes

- Now that we know how to create a native stack navigator with some routes and navigate between those routes, let's look at how we can pass data to routes when we navigate to them.
- There are two pieces to this:
  - Pass params to a route by putting them in an object as a second parameter to the navigation.navigate function:

    navigation.navigate('RouteName', { /* params go here */ })
  - Read the **params** in your screen component: **route.params.**

```
<Button
    title='Go to Profile'
    onPress={()=>navigation.navigate('Profile', {name: 'Meron'})}
/>
```

```
export default function Profile({navigation, route}) {
    const {name} =route.params;
  return (
    <View>
        <Text>{name} Profile</Text>
```

Initial params [1]

- You can also pass some initial **params** to a screen. If you didn't specify any params when navigating to this screen, the initial params will be used. They are also shallow merged with any params that you pass. Initial params can be specified with an **initialParams** prop:

```
<Stack.Screen
  name="Home"
  component={Home}
  options={{ title: 'My Home Page' }}
  initialParams={{ msg: 'Hello from main navigator' }}
  />
```

```
export default function Home({navigation, route}) {
    const { msg } = route.params;
```

- Passing additional props [2]
  - Sometimes we might want to pass additional props to a screen.
  - We can do that with 2 approaches:
    - Use React context and wrap the navigator with a context provider to pass data to the screens (recommended).
    - Use a render callback for the screen instead of specifying a component prop:

```
<Stack.Screen
  name="Home"
  options={{ title: 'My Home Page' }}
>
  {props => <Home {...props} msg='Hello from main navigator' />}
</Stack.Screen>
```

Tab navigation

- Possibly the most common style of navigation in mobile apps is tab-based navigation.
  - This can be tabs on the bottom of the screen or on the top below the header (or even instead of a header).
- First, you need to install the following package in your project:
  - **npm install @react-navigation/bottom-tabs**

Creating a simple tab navigation

- **Step:1- Import the following components**

```
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
```

- **Step:2 - Wrapping your app in NavigationContainer**
  - Now, we need to wrap the whole app in **NavigationContainer.**
- **Step:3 – Creating a tab navigation**

```
export default function App() {
  const Tab=createBottomTabNavigator();
  return (
```

- **Step:4 – Creating Tab Navigator**
- **Step:5 – Creating Home and Profile Screens, and import inside App**

```
export default function Home() {
  return (
    <View>
        <Text>This is Home</Text>
    </View>
  );
}
```

```
export default function Profile() {
  return (
    <View>
        <Text>This is Profile</Text>
    </View>
  );
}
```

```
import Home from './screens/Home';
import Profile from './screens/Profile';
export default function App() {
```

Step:6 – Creating Tab Screens

```
export default function App() {
  const Tab=createBottomTabNavigator();
  return (
    <NavigationContainer>
      <Tab.Navigator>
        <Tab.Screen name="Home" component={Home}/>
        <Tab.Screen name="Profile" component={Profile}/>
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

Drawer navigation

- Common pattern in navigation is to use drawer from left (sometimes right) side for navigating between screens.
- First, you need to install the following package in your project:
  - **npm install @react-navigation/drawer**
- Dependency installation
  - **npm install react-native-gesture-handler react-native-reanimated**
- To finalize installation of react-native-gesture-handler, add the following at the **top** (make sure it's at the top and there's nothing else before it) of your entry file, such as index.js or App.js:
  - **import 'react-native-gesture-handler';**

- The Drawer Navigator supports both Reanimated 1 and Reanimated 2. If you want to use Reanimated 2, make sure to configure it following the
  - **Babel plugin** : Add Reanimated's babel plugin to your babel.config.js:

```
module.exports = function(api) {
  api.cache(true);
  return {
    presets: ['babel-preset-expo'],
    plugins: [
        'react-native-reanimated/plugin',
    ],
  };
};
```

Creating a simple drawer navigation

- **Step:1- Import the following components**

```
import { NavigationContainer } from '@react-navigation/native';
import { createDrawerNavigator } from '@react-navigation/drawer';
```

- **Step:2 - Wrapping your app in NavigationContainer**
  - Now, we need to wrap the whole app in **NavigationContainer.**
- **Step:3 – Creating a Drawer navigation**

```
export default function App() {
  const Drawer=createDrawerNavigator();
  return (
```

- **Step:4 – Creating Drawer Navigator**
- **Step:5 – Creating Home and Profile Screens, and import inside App**

```
export default function Home() {
  return (
    <View>
        <Text>This is Home</Text>
    </View>
  );
}
```

```
export default function Profile() {
  return (
    <View>
        <Text>This is Profile</Text>
    </View>
  );
}
```

```
import Home from './screens/Home';
import Profile from './screens/Profile';
export default function App() {
```

- **Step:6 – Creating Drawer Screens**

```
export default function App() {
  const Drawer=createDrawerNavigator();
  return (
    <NavigationContainer>
      <Drawer.Navigator>
        <Drawer.Screen name="Home" component={Home}/>
        <Drawer.Screen name="Profile" component={Profile}/>
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

Exercise : Group Work

- Go to the site https://sampleapis.com/
  - **Sample APIs**
    - A simple, no fuss, no mess, no auth playground for learning RESTful or GraphQL APIs.
- Select any of the API List
  - Avators
  - BaseBall
  - Beers
  - FakeBank . . .
- **Design appropriate UI** for displaying the data

React Native Storage: Persisting data

- When building React Native applications, there are several ways to persist data, with each having its own strong advantage. The most popular ways to persist data in our React Native application are
  - AsyncStorage
  - React Native SQLite 2
  - Realm
  - React Native Local MongoDB

Persist data using **AsyncStorage**

- It is a **simple, asynchronous, unencrypted** by default module that allows you to **persist data offline in React Native apps.**
- The persistence of data is done in a **key-value storage system.**
  - Persisting data in a mobile app has benefits such as when the user restarts the app, the data or the setting variables being available to the user in the state they left before closing the app.
- **Security**
  - AsyncStorage encrypts none of the data saved. All objects are stored as strings and must be serialized before they can be saved and likewise be de-serialized before retrieval.

- We can perform three main actions with AsyncStorage: Set, Get, and Delete:
  - The **setItem()** method saves data to the AsyncStorage and allows a key and a value. Here, the key is a string that the data or value is assigned to:
  - The **getItem()** method allows us to get data back from AsyncStorage by using the key the data was saved as.
  - The **removeItem()** method removes data from AsyncStorage by using the key to delete the stored data:
  - The **clear()** method deletes or removes all the data from AsyncStorage. It does not need a key because it deletes all the stored data:

- **Installing the community module**
  - **npm install @react-native-async-storage/async-storage**
- **Usage**
  - **Async Storage** can **only store string data,** so in order to store object data you need to serialize it first.
    - For data that can be serialized to JSON you can use
      - **JSON.stringify()** when **saving** the data and
      - **JSON.parse()** when **loading** the data.
  - **Importing**
    - **import AsyncStorage from '@react-native-async-storage/async-storage';**

Storing Data

- **setItem()** is used both to **add new data** item (when no data for given key exists), and to **modify existing item** (when previous data for given key exists).
- **Storing string value**

```
const storeData = async (value) => {
  try {
    await AsyncStorage.setItem('@storage_Key', value)
  } catch (e) {
    // saving error
  }
}
```

- **Storing object value**

```
const storeData = async (value) => {
  try {
    const jsonValue = JSON.stringify(value)
    await AsyncStorage.setItem('@storage_Key', jsonValue)
  } catch (e) {
    // saving error
  }
}
```

Reading data

- **getItem** returns a promise that either resolves to stored value when data is found for given key, or returns null otherwise.
- **Reading string value**

```
const getData = async () => {
  try {
    const value = await AsyncStorage.getItem('@storage_Key')
    if(value !== null) {
      // value previously stored
    }
  } catch(e) {
    // error reading value
  }
}
```

Reading object value

```
const getData = async () => {
  try {
    const jsonValue = await AsyncStorage.getItem('@storage_Key')
    return jsonValue != null ? JSON.parse(jsonValue) : null;
  } catch(e) {
    // error reading value
  }
}
```

Persist data using **SQlite**

- It's very common for developers to use SQLite, a C-language library, as the data-store in mobile applications.
  - **SQLite** is an **open-source SQL database** that stores data to a text file on a device. It supports all the **relational database features**.
  - In order to access this database, we don't need to establish any kind of connections for it like JDBC, ODBC.
    - We only have to define the SQL statements for creating and updating the database. Access to it involves accessing the file system. This can be slow.
    - Therefore, it is recommended to perform database operations asynchronously.

- **Installing the community module**
  - npm install --save react-native-sqlite-storage
- **Database Setup**
  - While working with React Native it provides two option:
    - You can make the database in runtime.
    - You can use the pre-populated database so that you can start working directly.

- **How to Use react-native-sqlite-storage?**
  - **Importing**
    - **import SQLite from ‘react-native-sqlite-storage’**
  - **Open the database**
    - **let db = SQLite.openDatabase({ name: 'UserDatabase.db' });**
  - **Basic operations**
    - Creating table
    - Insert record
    - Update record
    - Delete record
    - Select record

- Now, whenever you need to make some database call you can **use db variable** to execute the database query.

```
db.transaction(function(txn) {
    txn.executeSql(
        query,  //Query to execute as prepared statement
        argsToBePassed[],  //Argument to pass for the prepared statement
        function(tx, res) {}  //Callback function to handle the result
    );
});
```

Thank You

Questions?